

Diáknyilvántartás

Feladat kiírás

Szoftver-fejlesztési képzés – Gyakorló feladat

A feladat célja

Implementálj egy teljes és működő diáknyilvántartó rendszert, amely konzolos, grafikus, szerver- és webes rétegből is áll. Az adatok forrása a teljesiskolairiport.csv fájl, illetve az adatbázist létrehozó SQL szkript.

1. C# konzolos feladat

Készíts egy konzolos alkalmazást, amely egy diáknyilvántartást valósít meg. Hozz létre egy új C# .NET konzolos projektet „DiakNyilvantartas” néven, és építs be az alábbi funkciókat.

Adatforrás

Másold a teljesiskolairiport.csv fájlt a projekt gyökerébe, és olvasd be a tartalmát program indításkor. Hozz létre egy Jegy osztályt, amely tartalmazza: diák neve, osztálya, tantárgya, jegye, tanár neve, dátum.

Alapvető statisztikák – Összegzés, Megszámlálás, Átlag

1 Összegzés

Számold ki az összes rögzített érdemjegy összegét!

2 Átlag

Számold ki az összes jegy átlagát a teljes adatsoron!

3 Megszámlálás

Hány darab jegyet rögzítettek a „11.B” osztály számára?

4 Szűrt átlag

Mennyi a „Matematika” tantárgyból szerzett jegyek átlaga?

5 Tanári statisztika

Hány jegyet osztott ki összesen „Vass Edit”?

Szélsőérték keresés – Minimum, Maximum

6 Maximum

Ki kapta a legutolsó jegyet az időrend alapján (legkésőbbi dátum)?

7 Minimum

Melyik a legkorábbi dátum, amikor jegyet rögzítettek?

8 Komplex maximum

Határozd meg, melyik diák kapta a legtöbb 5-öst!

9 Legszigorúbb tanár

Melyik tanárnál a legalacsonyabb a jegyek átlaga?

10 Legjobb diák

Ki az a diák, akinek a legmagasabb a tanulmányi átlaga?

Eldöntés és Keresés

11 Eldöntés

Van-e olyan diák, aki „Kémia” tárgyból 5-öst kapott? (Igen/Nem)

12 Keresés

Találd meg az első olyan bejegyzést, ahol a jegy 1-es! Írd ki a diák nevét és a tantárgyat!

13 Eldöntés

Minden diák kapott legalább egyszer 3-asnál jobb jegyet?

14 Keresés – szökőnap

Van-e olyan bejegyzés, amely február 29-re esik (szökőnap)?

Kiválogatás és Csoportosítás

15 Kiválogatás

Gyűjtsd ki egy listába az összes olyan diák nevét (ismétlődés nélkül), aki „Informatika” órát hallgat!

16 Negatív kiválogatás

Listázd ki az összes olyan bejegyzést (Név, Tárgy, Jegy), ahol a diák megbukott (1-est kapott)!

17 Csoportosítás

Számold meg, hány darab 5-ös, 4-es, 3-as, 2-es és 1-es van az összesített adatokban!

18 Osztály statisztika

Melyik osztályban született több jegy: a 10.A-ban vagy a 11.B-ben?

Összetett algoritmusok

19 Többszörös feltétel

Keresd meg azt a diákot, aki a legtöbb különböző tantárgyból kapott jegyet!

20 Dátum szűrés

Listázd ki azokat a diákokat, akik március hónapban kaptak jegyet!

21 Tanári rangsor

Rangsorold a tanárokat a kiosztott jegyek átlaga alapján, és írd ki a top 3 tanárt!

22 Diák rangsor

Rangsorold a diákokat a tanulmányi átlaguk alapján, és írd ki a top 5 diákot!

23 Tantárgyi rangsor

Rangsorold a tantárgyakat a kiosztott jegyek átlaga alapján, és írd ki a top 3 tantárgyat!

Megvalósítási javaslatok

Hozz létre egy Jegy osztályt a diák neve, osztálya, tantárgya, jegye, tanár neve és dátum mezőkkel. Használj List<Jegy> tárolót, és implementáld a funkciókat külön metódusokban. Az algoritmusokat LINQ használata nélkül, ciklus alapon valósítsd meg.

2. C# grafikus (GUI) feladat

Készíts egy Windows Forms vagy WPF alkalmazást, amely a diáknyilvántartó rendszer grafikus felületét valósítja meg.

1 Adatbevitel

Tedd lehetővé új diákok és jegyek hozzáadását egy űrlapon keresztül.

- Diák neve, osztálya, tantárgya
- Jegy értéke (1–5)
- Tanár neve és a bejegyzés dátuma

2 Adatmegjelenítés

Egy DataGridView-ben vagy ListView-ben jelenítsd meg az összes rögzített adatot.

3 Statisztikák

Egy külön panelen vagy ablakban jelenítsd meg a konzolos feladatban megfogalmazott statisztikákat és rangsorokat.

4 Szűrés és keresés

Tedd lehetővé a diákok és jegyek szűrését és keresését különböző kritériumok alapján:

- Név szerinti szűrés
- Tantárgy szerinti szűrés
- Jegy értéke alapján szűrés

5 Adatmentés és betöltés

Tedd lehetővé az adatok mentését fájlba (CSV vagy JSON formátumban) és visszatöltését onnan.

6 Felhasználói élmény

Az alkalmazás legyen felhasználóbarát, könnyen navigálható és esztetikus megjelenésű.

- Logikus menüstruktúra

- Hibaüzenet megjelenítése érvénytelen bevitel esetén
- Gyártó OOP elvek követése (osztályok, metódusok szeparálása)

3. Back-end fejlesztés (RESTful API)

Készíts egy RESTful API-t ASP.NET Core keretrendszerrel, amely a diáknyilvántartó rendszer szerver oldalát valósítja meg.

Adatbázis

Az adatbázist létrehozó SQL szkript: ./forrasok/diaknyilvatartas.sql

Kötelező végpontok

Metódus	Végpont (URL)	Leírás
GET	/students	Visszaadja az összes diák listáját
GET	/students/{id}	Visszaadja egy adott diák adatait azonosító alapján
POST	/students	Új diák hozzáadása az adatbázishoz
PUT	/students/{id}	Egy adott diák adatainak frissítése azonosító alapján
DELETE	/students/{id}	Egy adott diák törlése az adatbázisból

Követelmények

- Megfelelő HTTP állapotkód használata (200, 201, 404, 400, 500)
- JSON formátumú válaszok
- Bemeneti adatok validálása POST és PUT kéréseknél
- Hibás kérés esetén (pl. nem létező id) 404-es válasz
- Az API-t Entity Framework Core segítségével csatlakoztasd az adatbázishoz

Példa válasz – GET /students

```
[
  {
    "id": 1,
    "nev": "Kovács János",
    "osztaly": "11.B",
    "tantargy": "Matematika",
    "jegy": 4,
    "tanar": "Vass Edit",
    "datum": "2025-03-15"
  },
  ...
]
```


4. Front-end fejlesztés (React web alkalmazás)

Készíts egy egyszerű web alkalmazást React-ben, amely a fenti RESTful API-t használja a diáknyilvántartás kezelésére.

1 Diákok listázása

Jelenítsd meg az összes diák nevét és adatait egy táblázatban (Bootstrap 5 – table komponens).

2 Diák hozzáadása

Tedd lehetővé új diákok hozzáadását egy űrlapon keresztül (POST /students).

- Az űrlap tartalmazza az összes kötelező mezőt
- Sikeres hozzáadás után frissítődjön a lista
- Hibás bevétel esetén jelenítsen meg értelmes hibaüzenet

3 Diák szerkesztése

Tedd lehetővé egy adott diák adatainak frissítését egy szerkesztő űrlapon keresztül (PUT /students/{id}).

4 Diák törlése

Tedd lehetővé egy adott diák törlését egy gomb megnyomásával (DELETE /students/{id}).

- Megerősítő párbeszéd panel megjelenítése törlés előtt
- Törlés után frissítődjön a lista automatikusan

Általános elvárások a webes alkalmazással szemben

Az alkalmazás legyen reszponzív (Bootstrap 5). Az API-hívásokhoz használj axios könyvtárat. Gondoskodj megfelelő töltési és hibaállapotok kezeléséről. Implementálj navigációt React Router segítségével.